



Predicting Annual Income

By Nicholas Barrale, Paramjit Kaur

Dataset

- Found on Kaggle
- Determines whether an adult makes over \$50k/yr
- Sample of over 48,000 adults
- 14 different attributes are listed (age, education, gender, etc..)
- Includes both numerical and categorical features

age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

Our Goal



Target Variable: Annual Income



Use machine learning models to predict whether or not an adult exceeds \$50k/yr



Binary Classification problem ($\leq 50k$ or $> 50k$)



Use Logistic Regression and Random Forest



Logistic Regression

- Supervised machine learning algorithm for classification
- Predicts probability of belonging to a class
- Commonly used for binary outcomes
- Suitable for income prediction with two categories

Logistic Regression

- Encode target variable(income)
- $\leq 50k$ is 0
- $> 50k$ is 1

Encode Target Variable

```
#encode the target variable 'income.'  $\leq 50K$  will be 0,  $> 50K$  will be 1
le = LabelEncoder()
df_processed['income_encoded'] = le.fit_transform(df_processed['income'])

print("Original income categories:", le.classes_)
print("Encoded income unique values:", df_processed['income_encoded'].unique())

... Original income categories: ['  $\leq 50K$ ' '  $> 50K$ ']
    Encoded income unique values: [0 1]
```

Logistic Regression Cont.

Split data with a test size of .2

One-Hot Encode Categorical Features and Split Data

```
x = df_processed.drop(['income', 'income_encoded'], axis=1)
y = df_processed['income_encoded']

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough'
)

X_processed = preprocessor.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=0.2, random_state=42, stratify=y)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

... Shape of X_train: (26048, 105)
Shape of X_test: (6513, 105)
Shape of y_train: (26048,)
Shape of y_test: (6513,)

Logistic Regression Cont.

- Tune hyperparameters
- Train the model with grid_search.fit with X_train and y_train

Hyperparameters and Train the Model

```
log_reg = LogisticRegression(solver='liblinear', random_state=42,

param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100],
    'penalty': ['l1', 'l2']

from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(estimator=log_reg, param_grid=param_grid)
grid_search.fit(X_train, y_train)

print(f"Best hyperparameters: {grid_search.best_params_}")

print(f"Best cross-validation accuracy: {grid_search.best_score_:.3f}")

est_log_reg = grid_search.best_estimator_

# Fitting 5 folds for each of 12 candidates, totalling 60 fits
est hyperparameters: {'C': 1, 'penalty': 'l1'}
est cross-validation accuracy: 0.8496
```

Logistic Regression Performance Evaluation

Model was fairly accurate with a score of 0.86



```
y_pred = best_log_reg.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print(f"Test Accuracy: {accuracy:.4f}")
print("\nClassification Report:\n", report)
print("\nConfusion Matrix:\n", conf_matrix)
```

... Test Accuracy: 0.8552

```
Classification Report:
              precision    recall  f1-score   support

     0       0.88      0.93      0.91      4945
     1       0.74      0.62      0.67      1568

 accuracy         0.85
 macro avg        0.81
 weighted avg     0.85
```

```
Confusion Matrix:
[[4605  340]
 [ 603  965]]
```

Sample of Actual vs. Predicted Income:

	Actual_Encoded	Predicted_Encoded	Actual_Income	Predicted_Income
0	0	0	<=50K	<=50K
1	0	0	<=50K	<=50K
2	1	1	>50K	>50K
3	1	1	>50K	>50K
4	0	0	<=50K	<=50K
5	0	0	<=50K	<=50K
6	0	0	<=50K	<=50K
7	0	1	<=50K	>50K
8	0	0	<=50K	<=50K
9	0	0	<=50K	<=50K



Random Forest

- Ensemble learning method using multiple decision trees
- Reduces overfitting by combining multiple models
- Captures complex, non-linear relationships in data
- Combines predictions to improve accuracy

Random Forest Implementation

- Split data using test size of 0.2
- Applied hyperparameter tuning using GridSearchCV
- Tuned parameters:
 - n_estimators: 100, 200
 - max_depth: 10, 20, None
 - min_samples_split: 2, 5

```
rf = RandomForestClassifier(random_state=42)

param_grid_rf = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5]
}

grid_search_rf = GridSearchCV(
    estimator=rf,
    param_grid=param_grid_rf,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid_search_rf.fit(X_train, y_train)

print(f"Best RF hyperparameters: {grid_search_rf.best_params_}")
print(f"Best RF cross-validation accuracy: {grid_search_rf.best_score_:.4f}")

best_rf = grid_search_rf.best_estimator_
```

Fitting 5 folds for each of 12 candidates, totalling 60 fits
Best RF hyperparameters: {'max_depth': 20, 'min_samples_split': 5, 'n_estimators': 200}
Best RF cross-validation accuracy: 0.8626

Random Forest Performance Evaluation

- Model achieved an accuracy of 0.868 - 86.8%
- Higher accuracy than Logistic Regression
- Better at capturing complex patterns in the data
- Produced fewer classification errors

Predict and Evaluate Model Performance

```
rf_pred = best_rf.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)
rf_report = classification_report(y_test, rf_pred)
rf_conf_matrix = confusion_matrix(y_test, rf_pred)

print(f"Random Forest Test Accuracy: {rf_accuracy:.4f}")
print("\nRandom Forest Classification Report:\n", rf_report)
print("\nRandom Forest Confusion Matrix:\n", rf_conf_matrix)
```

Random Forest Test Accuracy: 0.8683

Random Forest Classification Report:				
	precision	recall	f1-score	support
0	0.89	0.95	0.92	4945
1	0.79	0.62	0.69	1568
accuracy			0.87	6513
macro avg	0.84	0.78	0.80	6513
weighted avg	0.86	0.87	0.86	6513

Random Forest Confusion Matrix:

```
[[4685 260]
 [ 598 970]]
```

Model Comparison

- Logistic Regression Accuracy: 85.5%
- Random Forest Accuracy: 86.8%
- Random Forest shows improved predictive performance
- Indicates presence of non-linear patterns in data
- Trade-off:
 - Logistic Regression → simple & interpretable
 - Random Forest → more complex but more accurate

Compare Actual vs Predicted Results

```
rf_results_df = pd.DataFrame({
    'Actual_Encoded': y_test,
    'Predicted_Encoded': rf_pred
})

rf_results_df = rf_results_df.reset_index(drop=True)

rf_results_df['Actual_Income'] = le.inverse_transform(rf_results_df['Actual_Encoded'])
rf_results_df['Predicted_Income'] = le.inverse_transform(rf_results_df['Predicted_Encoded'])

print("\nSample of Actual vs. Predicted Income (Random Forest):")
display(rf_results_df.head(10))
```

Sample of Actual vs. Predicted Income (Random Forest):

	Actual_Encoded	Predicted_Encoded	Actual_Income	Predicted_Income
0	0	0	<=50K	<=50K
1	0	0	<=50K	<=50K
2	1	1	>50K	>50K
3	1	1	>50K	>50K
4	0	0	<=50K	<=50K
5	0	0	<=50K	<=50K
6	0	0	<=50K	<=50K
7	0	0	<=50K	<=50K
8	0	0	<=50K	<=50K
9	0	0	<=50K	<=50K

```
comparison_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [accuracy, rf_accuracy]
})

print(comparison_df)
```

	Model	Accuracy
0	Logistic Regression	0.855213
1	Random Forest	0.868263

Conclusion

- Both models performed well
- Random Forest achieved highest accuracy
- Machine learning can predict income effectively
- Future improvements:
 - Feature selection
 - More models
 - Data balancing

Thanks for listening
